DISASTER TWEET CLASSIFICATION

Name: NEHA PRADHAN

Company: NextHikes IT solutions

Date: 18.03.2026

Introduction

In the modern digital age, social media platforms—especially Twitter—have become powerful channels for real-time information sharing. During emergencies such as earthquakes, floods, fires, and other disasters, people often post updates instantly, making Twitter a valuable source for early warning signals and situational awareness.

However, a major challenge lies in distinguishing between tweets that describe actual disasters and those that use similar words in a non-literal or casual context. For example, phrases like *"this exam is a disaster"* or *"my phone exploded"* do not represent real emergencies. This ambiguity makes manual monitoring inefficient and error-prone.

To address this challenge, this project leverages Natural Language Processing (NLP) and Machine Learning techniques to automatically classify tweets as disaster-related or non-disaster-related. By analyzing textual patterns, context, and linguistic features, the model aims to provide accurate and reliable predictions.

This system not only demonstrates the practical application of NLP in real-world scenarios but also highlights its potential in supporting disaster management systems, enabling faster response times, and improving public safety through automated information filtering.

Objective

The primary objective of this project is to design and develop an intelligent Natural Language Processing (NLP) system capable of accurately classifying tweets as disaster-related or non-disaster-related. This project aims to:

- Build a robust machine learning model that can understand the context and intent behind textual data
- Achieve high accuracy and reliability in distinguishing real disaster information from casual or metaphorical usage
- Implement effective text preprocessing and feature engineering techniques to enhance model performance
- Analyze and evaluate the model using standard performance metrics such as accuracy, precision, recall, and F1-score
- Develop a user-friendly web application that allows real-time prediction of tweets

Dataset Description

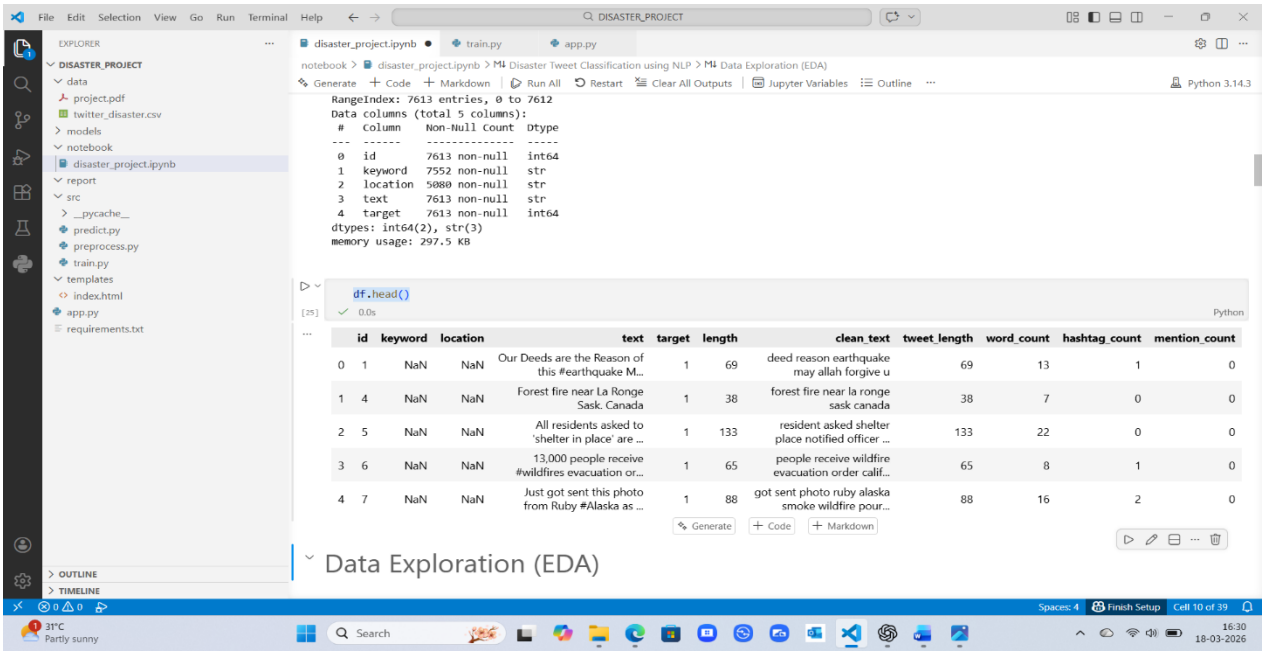
The dataset used in this project is **twitter_disaster.csv**, which consists of approximately 10,000 manually labeled tweets collected from Twitter. This dataset is specifically designed for training machine learning models to identify disaster-related content.

Each tweet in the dataset has been carefully annotated to indicate whether it refers to a real disaster or not, making it suitable for supervised learning tasks.

◈ Dataset Features:

- **text**: Contains the actual tweet content posted by users
- **target**: Binary label indicating the nature of the tweet
  - 0 → Non-Disaster Tweet
  - 1 → Disaster Tweet

The dataset includes a wide variety of tweet styles, including informal language, abbreviations, hashtags, mentions, and slang. This diversity makes the classification task more challenging and realistic, as the model must understand context rather than relying solely on keywords. By working with this dataset, the model learns to differentiate between literal and figurative usage of disaster-related terms, thereby improving its ability to make accurate predictions in real-world scenarios. Ultimately, the goal is to demonstrate how machine learning can be applied to solve real-world problems by enabling faster and more efficient disaster detection through automated text classification.



Data Preprocessing

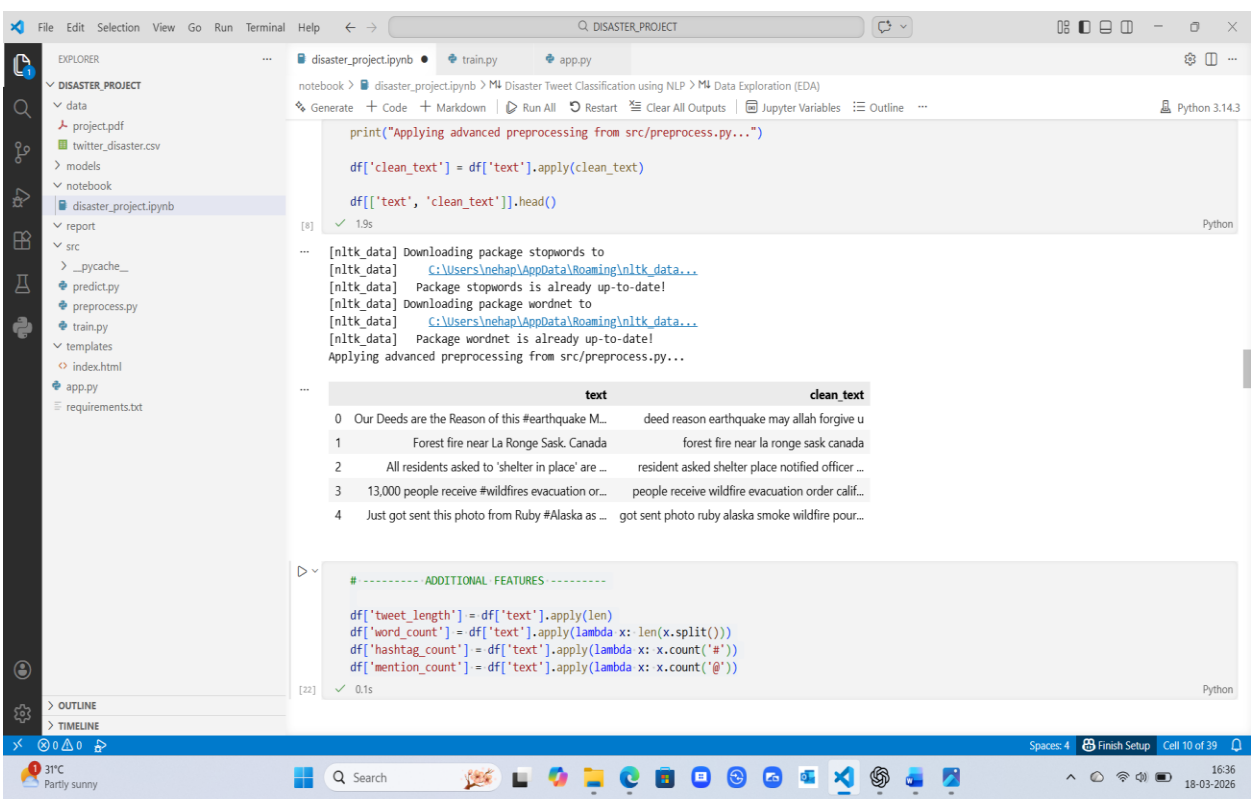
Data preprocessing is a crucial step in NLP.

The following steps were performed:

- Conversion of text to lowercase
- Removal of URLs
- Removal of special characters
- Removal of stopwords
- Lemmatization of words

Example Code:

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^a-zA-Z]", " ", text)
    words = text.split()
    words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words]
    return " ".join(words)
```



Feature Engineering

Feature engineering converts text into numerical form.

◈ 6.1 TF-IDF Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) is used to convert text into numerical vectors.

- Captures importance of words
- Uses unigrams and bigrams
- Limits features to 10,000

◈ 6.2 Additional Features (Advanced)

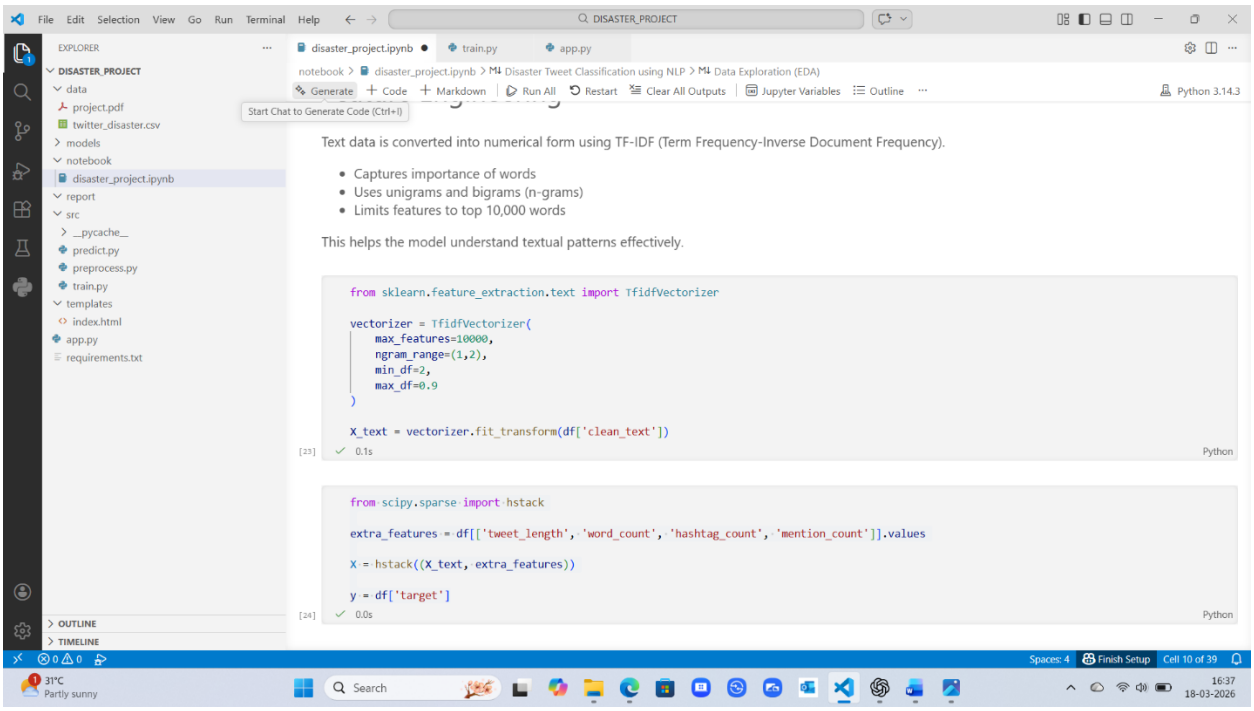
To improve model performance, extra features were added:

- **Tweet Length** → Total number of characters
- **Word Count** → Number of words
- **Hashtag Count** → Number of '#' symbols
- **Mention Count** → Number of '@' mentions

Code:

```
df['tweet_length'] = df['text'].apply(len)
df['word_count'] = df['text'].apply(lambda x: len(x.split()))
df['hashtag_count'] = df['text'].apply(lambda x: x.count('#'))
df['mention_count'] = df['text'].apply(lambda x: x.count('@'))
```

These features help capture additional contextual information.



Model Deployment

The trained model was deployed using Flask.

Steps:

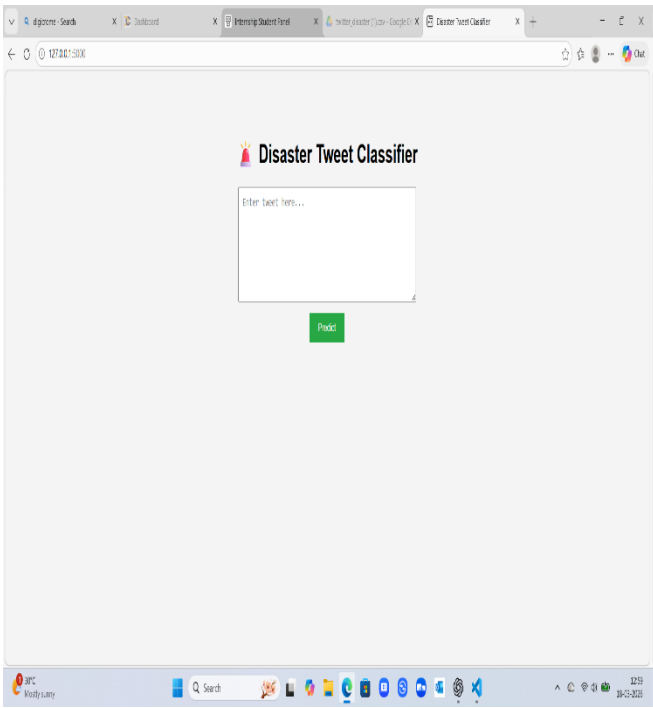
- Model saved using pickle
- Web interface created
- User inputs tweet
- Model predicts output

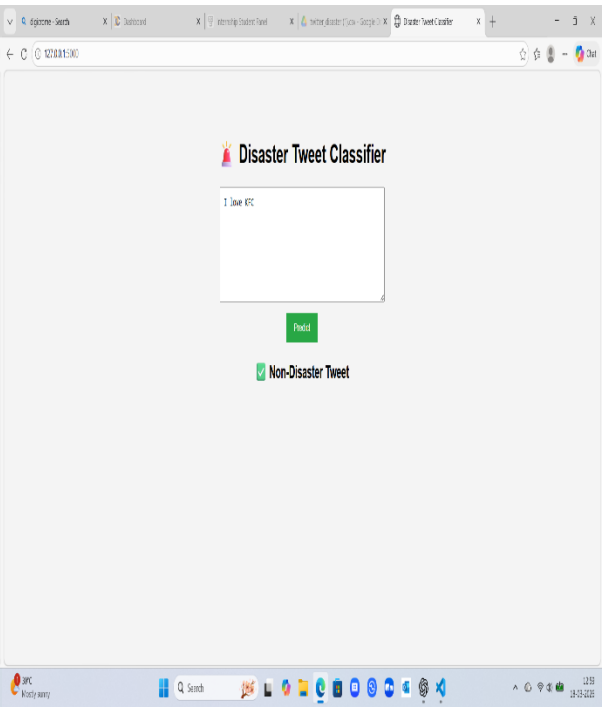
---

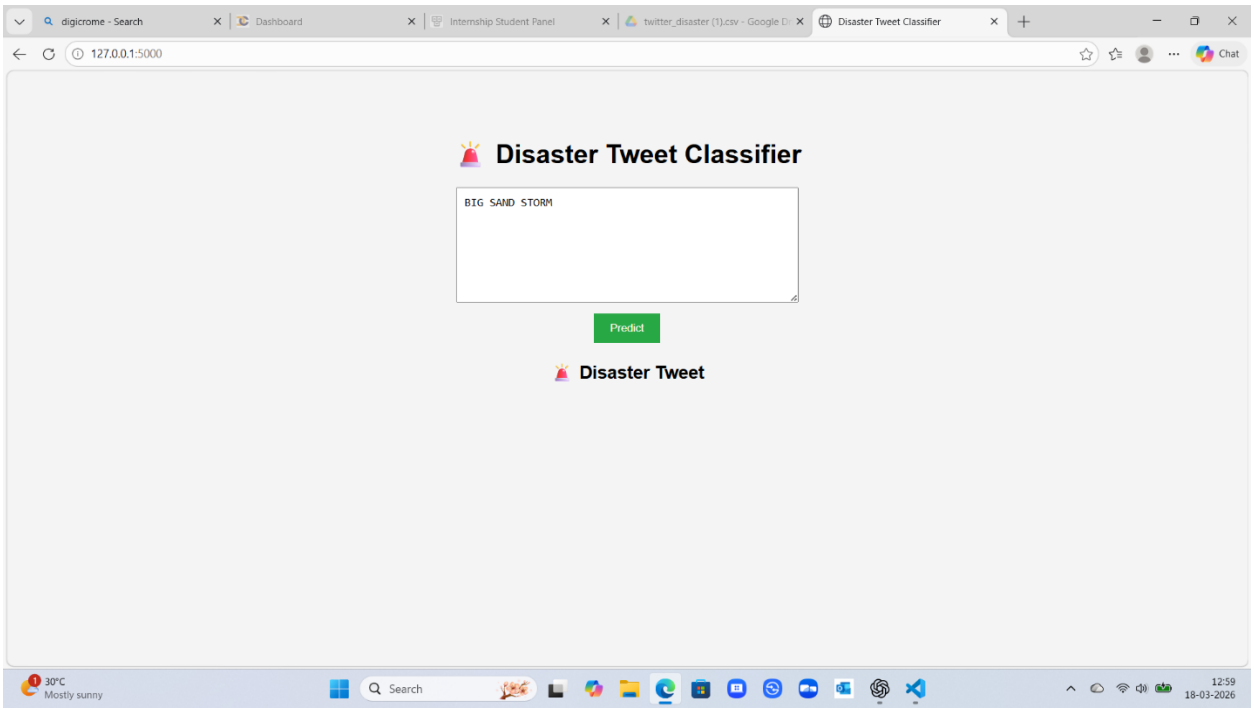
10. User Interface

A simple web interface was developed where:

- User enters tweet text
- Clicks predict button
- Result is displayed


Disaster Tweet Classifier
Enter tweet here...
Predict


Disaster Tweet Classifier
I love KFC
Predict
Non-Disaster Tweet


digicrome - Search
Dashboard
Internship Student Panel
twitter_disaster (1).csv - Google Dr
Disaster Tweet Classifier
127.0.0.1:5000
Chat
Disaster Tweet Classifier
BIG SAND STORM
Predict
Disaster Tweet
30°C
Mostly sunny
Search
12:59
18-03-2026

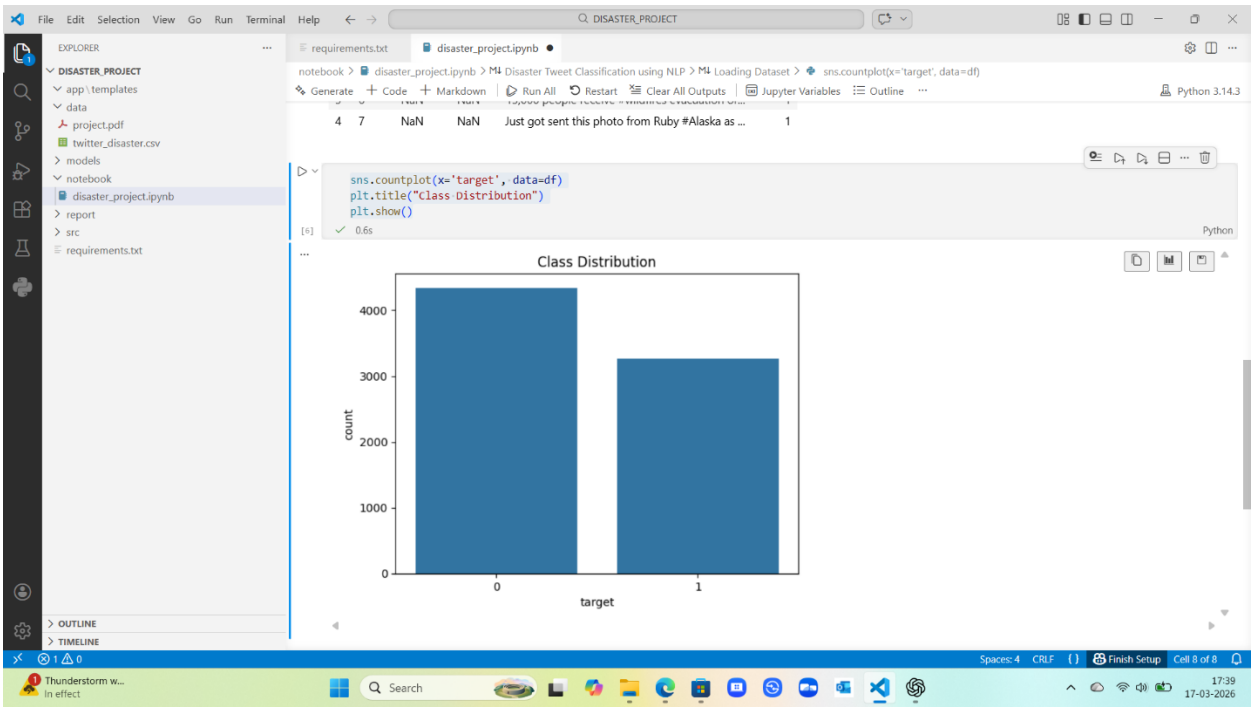
Conclusion

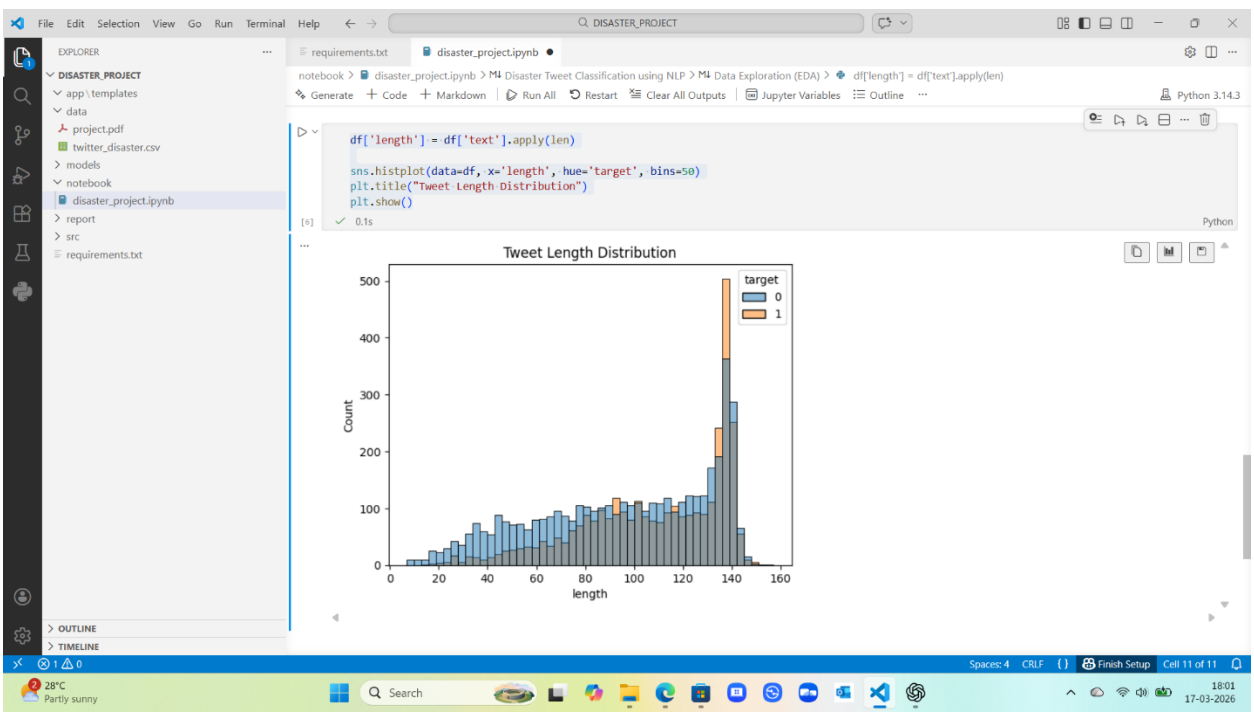
This project successfully demonstrates the use of NLP techniques to classify disaster-related tweets. The model performs well and can be further improved using advanced models like deep learning.

---

12. Future Scope

- Use Deep Learning (LSTM, BERT)
- Real-time Twitter API integration
- Improve feature engineering
- Deploy on cloud platforms


File Edit Selection View Go Run Terminal Help
DISASTER_PROJECT
EXPLORER
DISASTER_PROJECT
app\templates
data
project.pdf
twitter_disaster.csv
models
notebook
disaster_project.ipynb
report
src
requirements.txt
OUTLINE
TIMELINE
requirements.txt
disaster_project.ipynb
notebook > disaster_project.ipynb > Disaster Tweet Classification using NLP > Loading Dataset > sns.countplot(x='target', data=df)
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Python 3.14.3
4 7 NaN NaN Just got sent this photo from Ruby #Alaska as ... 1
sns.countplot(x='target', data=df)
plt.title("Class Distribution")
plt.show()
[6] 0.6s
Python
Class Distribution
count
target
0
1
0
1000
2000
3000
4000
Spaces: 4 CRLF Finish Setup Cell 8 of 8
Thunderstorm w...
In effect
Search
17:39
17-03-2026


File Edit Selection View Go Run Terminal Help
DISASTER_PROJECT
EXPLORER
DISASTER_PROJECT
app\templates
data
project.pdf
twitter_disaster.csv
models
notebook
disaster_project.ipynb
report
src
requirements.txt
OUTLINE
TIMELINE
requirements.txt
disaster_project.ipynb
notebook > disaster_project.ipynb > Disaster Tweet Classification using NLP > Data Exploration (EDA) > df['length'] = df['text'].apply(len)
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Python 3.14.3
df['length'] = df['text'].apply(len)
sns.histplot(data=df, x='length', hue='target', bins=50)
plt.title("Tweet Length Distribution")
plt.show()
[6] 0.1s
Python
Tweet Length Distribution
target
0
1
Count
length
0
100
200
300
400
500
0
20
40
60
80
100
120
140
160
Spaces: 4 CRLF Finish Setup Cell 11 of 11
28°C
Partly sunny
Search
18:01
17-03-2026

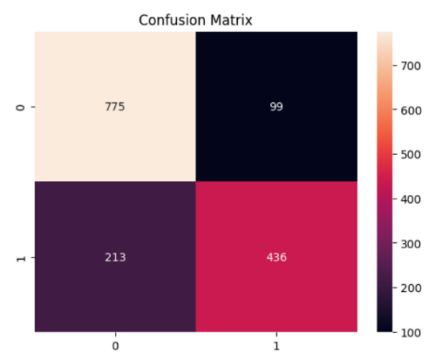
```python
results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    acc = accuracy_score(y_test, preds)
    results[name] = acc

    print(f"\n{name}")
    print("Accuracy:", acc)
```

```
Logistic Regression
Accuracy: 0.7951411687458962

Naive Bayes
Accuracy: 0.7964543663821405
```

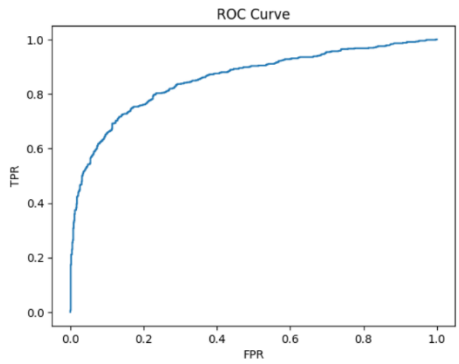
```python
pd.DataFrame(results.items(), columns=["Model", "Accuracy"])
```

	Model	Accuracy
0	Logistic Regression	0.795141
1	Naive Bayes	0.796454

```python
cm = confusion_matrix(y_test, preds)

sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()
```



```python
probs = best_model.predict_proba(X_test)[:,1]

fpr, tpr, _ = roc_curve(y_test, probs)

plt.plot(fpr, tpr)
plt.title("ROC Curve")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.show()
```




```python
# Import required libraries
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer


# Download required datasets (runs once)
nltk.download('stopwords')
nltk.download('wordnet')

# Initialize stopwords and lemmatizer
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()


# Step 1: Remove URLs
def remove_urls(text):
    return re.sub(r"http\S+", "", text)


# Step 2: Remove special characters
def remove_special_chars(text):
    return re.sub(r"[^a-zA-Z]", " ", text)


# Step 3: Tokenization
def tokenize(text):
    return text.split()


# Step 4: Remove stopwords
def remove_stopwords(words):
    return [w for w in words if w not in stop_words]


# Step 5: Lemmatization
```

```python
# Feature engineering
print("Vectorizing text...")
vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1,2))
X = vectorizer.fit_transform(df['clean_text'])
y = df['target']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
print("Training model...")
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Evaluation
print("\nEvaluating model...")
preds = model.predict(X_test)

accuracy = accuracy_score(y_test, preds)
print("Accuracy:", accuracy)
```

```
    accuracy                           0.80      1523
   macro avg       0.80      0.78      0.78      1523
weighted avg       0.80      0.80      0.79      1523


Confusion Matrix:
[[775  99]
 [213 436]]

Saving model...
✅ Model trained and saved successfully!
PS C:\Users\nehap\OneDrive\Desktop\DISASTER_PROJECT\src>
```

```python
import pickle

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from preprocess import clean_text


# Load dataset
print("Loading dataset...")
df = pd.read_csv("../data/twitter_disaster.csv")

# Apply preprocessing
print("Cleaning text...")
df['clean_text'] = df['text'].apply(clean_text)

# Feature engineering
print("Vectorizing text...")
vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1,2))
X = vectorizer.fit_transform(df['clean_text'])
```

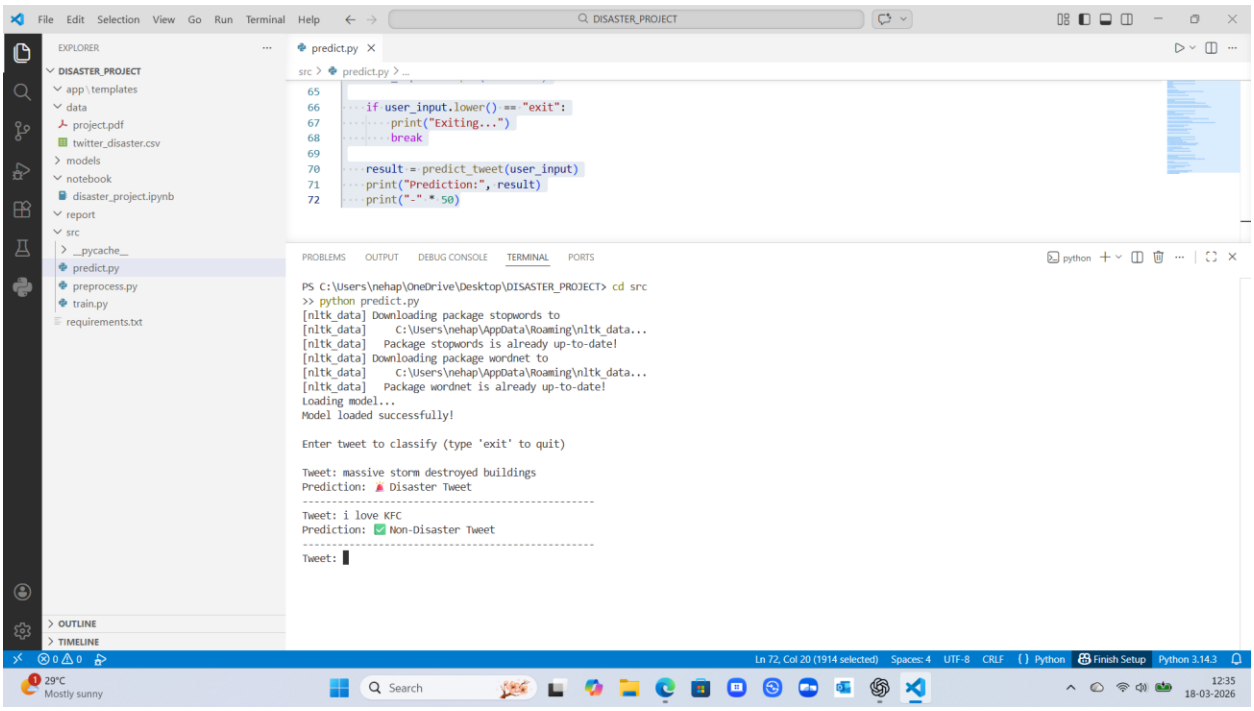
```
  File "<stdin>", line 1, in <module>
    pickle.dump(model, open("../models/best_model.pkl", "wb"))
                           ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: '../models/best_model.pkl'
>>> pickle.dump(vectorizer, open("../models/vectorizer.pkl", "wb"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    pickle.dump(vectorizer, open("../models/vectorizer.pkl", "wb"))
                            ~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: '../models/vectorizer.pkl'
>>> print("✅ Model trained and saved successfully!")
✅ Model trained and saved successfully!
>>>
```

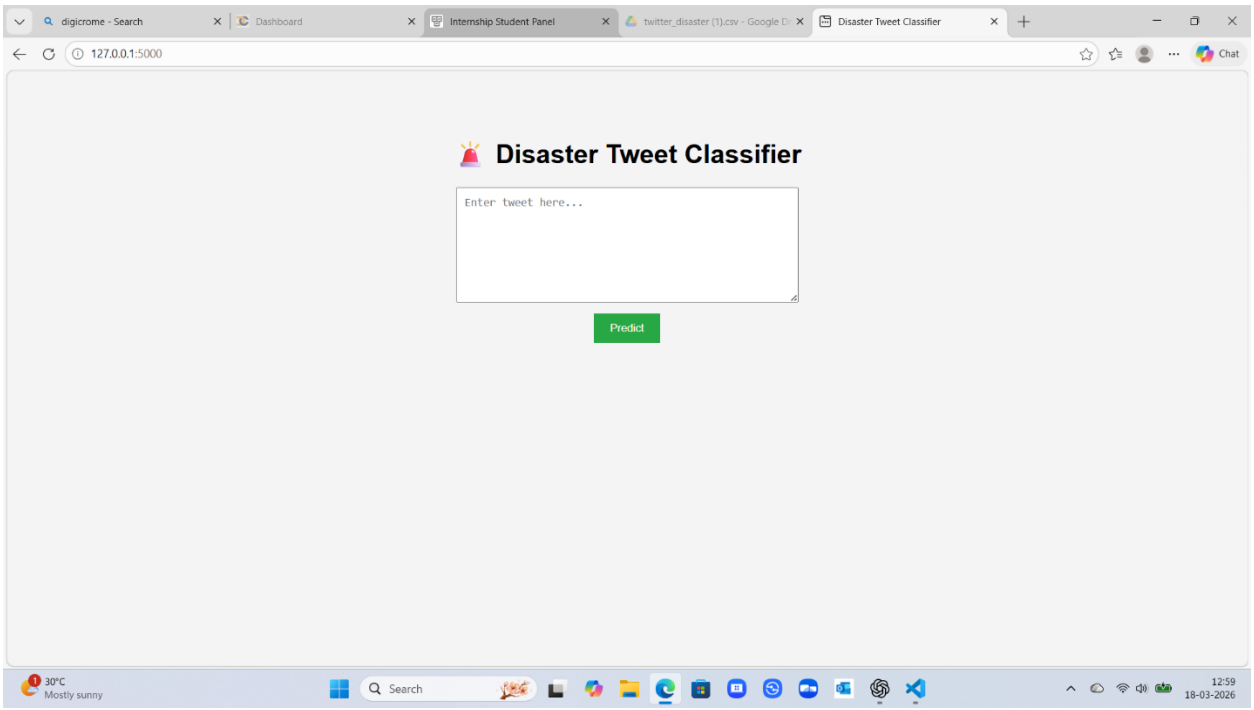
```python
print("\nAccuracy:", accuracy)
print("\nClassification Report:\n")
print(classification_report(y_test, preds))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, preds))


# ========================================
```

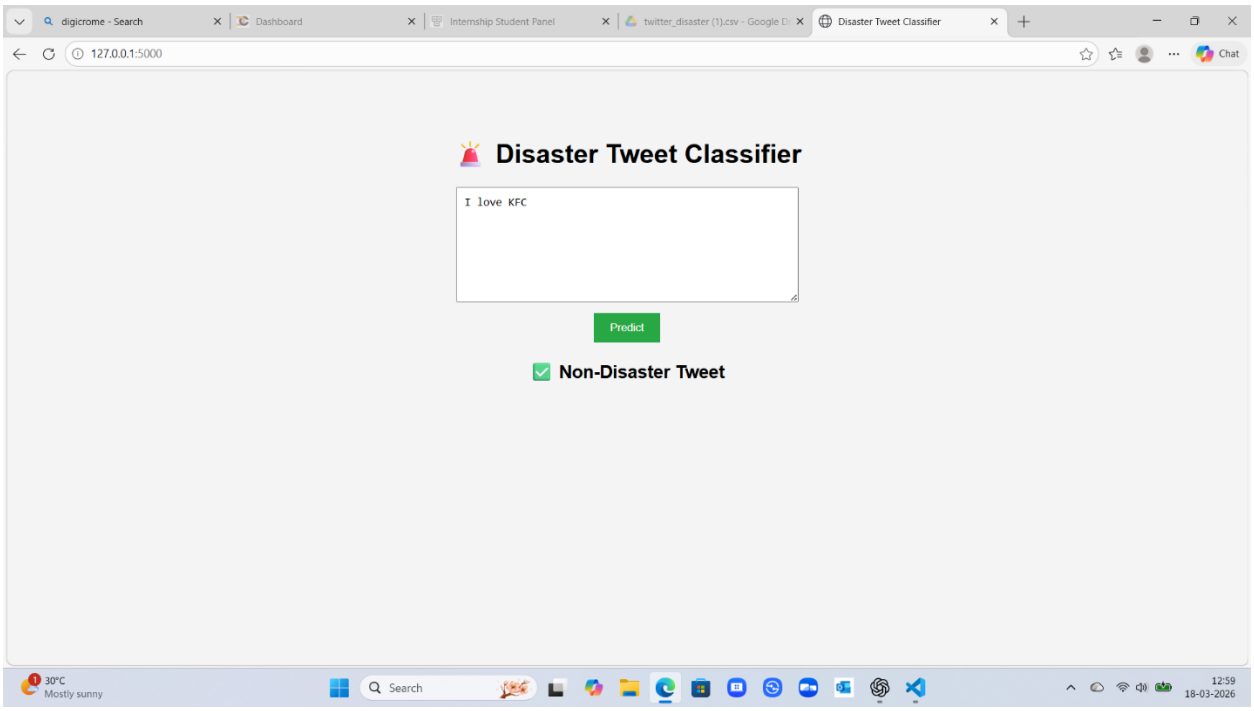
```
Dataset loaded successfully!
   id keyword location                                               text  target
0   1     NaN      NaN  Our Deeds are the Reason of this #earthquake M...       1
1   4     NaN      NaN             Forest fire near La Ronge Sask. Canada       1
2   5     NaN      NaN  All residents asked to 'shelter in place' are ...       1
3   6     NaN      NaN  13,000 people receive #wildfires evacuation or...       1
4   7     NaN      NaN  Just got sent this photo from Ruby #Alaska as ...       1

Cleaning text...
Text cleaning completed!

Vectorizing text...
Vectorization completed!

Splitting dataset...
Data split completed!

Training model...
Model training completed!

Evaluating model...

Accuracy: 0.7951411687458962

Classification Report:
```

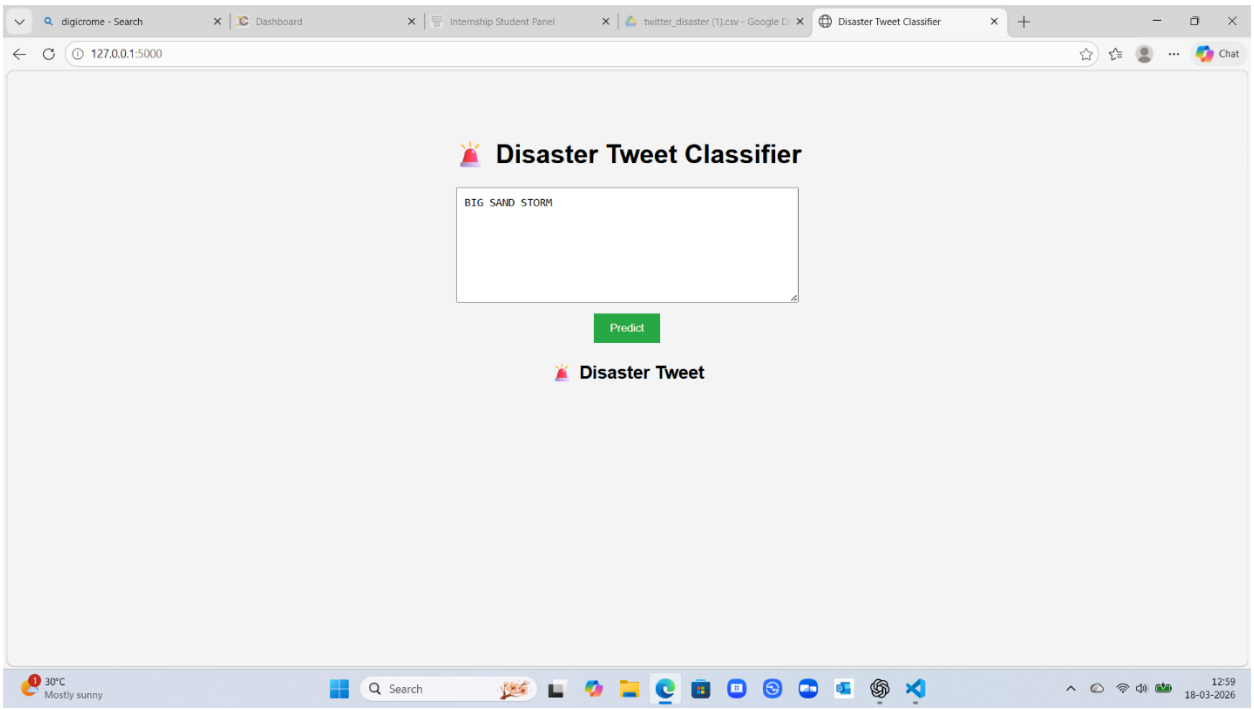

```python
print("\nAccuracy:", accuracy)
print("\nClassification Report:\n")
print(classification_report(y_test, preds))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, preds))


# ========================================
```


```
Accuracy: 0.7951411687458962

Classification Report:

              precision    recall  f1-score   support

           0       0.78      0.89      0.83       874
           1       0.81      0.67      0.74       649

    accuracy                           0.80      1523
   macro avg       0.80      0.78      0.78      1523
weighted avg       0.80      0.80      0.79      1523


Confusion Matrix:

[[775  99]
 [213 436]]

Saving model...
Model saved successfully!
Saved at: C:\Users\nehap\OneDrive\Desktop\DISASTER_PROJECT\models
PS C:\Users\nehap\OneDrive\Desktop\DISASTER_PROJECT\src>
```


DISASTER_PROJECT
predict.py
if user_input.lower() == "exit":
print("Exiting...")
break
result = predict_tweet(user_input)
print("Prediction:", result)
print("-" * 50)
PS C:\Users\nehap\OneDrive\Desktop\DISASTER_PROJECT> cd src
>> python predict.py
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\nehap\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\nehap\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
Loading model...
Model loaded successfully!
Enter tweet to classify (type 'exit' to quit)
Tweet: massive storm destroyed buildings
Prediction: Disaster Tweet
Tweet: i love KFC
Prediction: Non-Disaster Tweet
Tweet:


127.0.0.1:5000
Disaster Tweet Classifier
Enter tweet here...
Predict


digicrome - Search
Dashboard
Internship Student Panel
twitter_disaster (1).csv - Google Dr
Disaster Tweet Classifier
127.0.0.1:5000
Chat
Disaster Tweet Classifier
I love KFC
Predict
Non-Disaster Tweet
30°C
Mostly sunny
Search
12:59
18-03-2026


digicrome - Search
Dashboard
Internship Student Panel
twitter_disaster (1).csv - Google Dr
Disaster Tweet Classifier
127.0.0.1:5000
Chat
Disaster Tweet Classifier
BIG SAND STORM
Predict
Disaster Tweet
30°C
Mostly sunny
Search
12:59
18-03-2026

```
RangeIndex: 7613 entries, 0 to 7612
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   id        7613 non-null   int64
 1   keyword   7552 non-null   str
 2   location  5080 non-null   str
 3   text      7613 non-null   str
 4   target    7613 non-null   int64
dtypes: int64(2), str(3)
memory usage: 297.5 KB
```

```python
df.head()
```

[25] ✓ 0.0s Python

	id	keyword	location	text	target	length	clean_text	tweet_length	word_count	hashtag_count	mention_count
0	1	NaN	NaN	Our Deeds are the Reason of this #earthquake M...	1	69	deed reason earthquake may allah forgive u	69	13	1	0
1	4	NaN	NaN	Forest fire near La Ronge Sask. Canada	1	38	forest fire near la ronge sask canada	38	7	0	0
2	5	NaN	NaN	All residents asked to 'shelter in place' are ...	1	133	resident asked shelter place notified officer ...	133	22	0	0
3	6	NaN	NaN	13,000 people receive #wildfires evacuation or...	1	65	people receive wildfire evacuation order calif...	65	8	1	0
4	7	NaN	NaN	Just got sent this photo from Ruby #Alaska as ...	1	88	got sent photo ruby alaska smoke wildfire pour...	88	16	2	0

Data Exploration (EDA)

```python
print("Applying advanced preprocessing from src/preprocess.py...")

df['clean_text'] = df['text'].apply(clean_text)

df[['text', 'clean_text']].head()
```

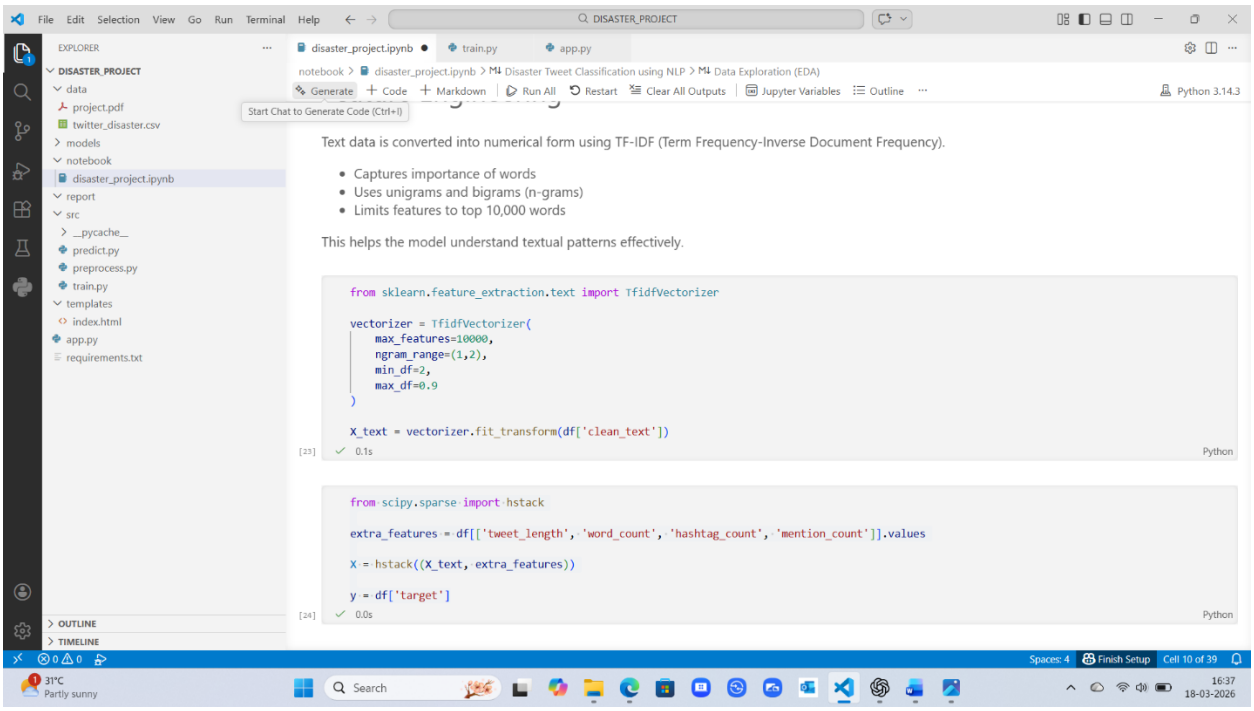
[8] ✓ 1.9s Python

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\nehap\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\nehap\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Applying advanced preprocessing from src/preprocess.py...
```

	text	clean_text
0	Our Deeds are the Reason of this #earthquake M...	deed reason earthquake may allah forgive u
1	Forest fire near La Ronge Sask. Canada	forest fire near la ronge sask canada
2	All residents asked to 'shelter in place' are ...	resident asked shelter place notified officer ...
3	13,000 people receive #wildfires evacuation or...	people receive wildfire evacuation order calif...
4	Just got sent this photo from Ruby #Alaska as ...	got sent photo ruby alaska smoke wildfire pour...

```python
# --------- ADDITIONAL FEATURES ---------

df['tweet_length'] = df['text'].apply(len)
df['word_count'] = df['text'].apply(lambda x: len(x.split()))
df['hashtag_count'] = df['text'].apply(lambda x: x.count('#'))
df['mention_count'] = df['text'].apply(lambda x: x.count('@'))
```

[22] ✓ 0.1s Python


File Edit Selection View Go Run Terminal Help
DISASTER_PROJECT
EXPLORER
DISASTER_PROJECT
data
project.pdf
twitter_disaster.csv
models
notebook
disaster_project.ipynb
report
src
__pycache__
predict.py
preprocess.py
train.py
templates
index.html
app.py
requirements.txt
OUTLINE
TIMELINE
disaster_project.ipynb
train.py
app.py
notebook > disaster_project.ipynb > Disaster Tweet Classification using NLP > Data Exploration (EDA)
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.3
Start Chat to Generate Code (Ctrl+I)
Text data is converted into numerical form using TF-IDF (Term Frequency-Inverse Document Frequency).
Captures importance of words
Uses unigrams and bigrams (n-grams)
Limits features to top 10,000 words
This helps the model understand textual patterns effectively.
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(
max_features=10000,
ngram_range=(1,2),
min_df=2,
max_df=0.9
)
X_text = vectorizer.fit_transform(df['clean_text'])
[23] 0.1s
Python
from scipy.sparse import hstack
extra_features = df[['tweet_length', 'word_count', 'hashtag_count', 'mention_count']].values
X = hstack((X_text, extra_features))
y = df['target']
[24] 0.0s
Python
Spaces: 4
Finish Setup
Cell 10 of 39
31°C
Partly sunny
Search
16:37
18-03-2026